

Git & GitHub

★ Git → free & open source version Control System
↳ tools that help to track changes in code.

- 1.) Track history
- 2.) help to collaborate

★ GitHub → website where we host repositories online.

★ Readme.md → markdown

★ commit → change

↳ internally for every change
first [add] then [commit]

→
green area → what you added
red area → what you deleted

★ Git = The actual tool that tracks code changes on your computer.

GitHub = The website where you store your Git repositories online & share them with others.

Git Bash = a terminal window on windows where you type Git commands.

★ Using Git :-

- 1.) Command line
- 2.) IDE / Code Editors (like Vs Code)
- 3.) Graphical User Interface (like GitKraken)

★ Configuring Orit :-

global / local

```
git config --global user.name "My Name"
```

git config --global user.email "someone@email.com".

★ Basic Commands :-

1.) Clone - Cloning a repository on our local machine

git clone <link>

ls → list files ls -la → list hidden files

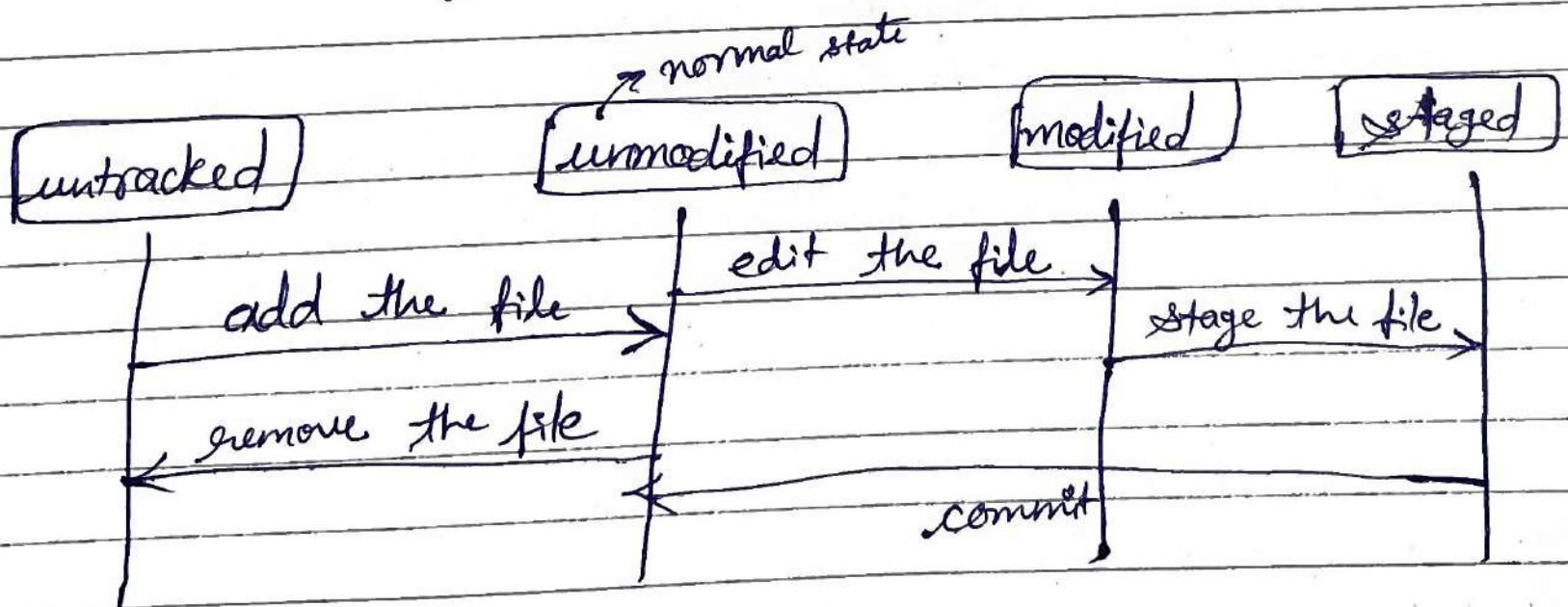
every git file has '.git' hidden file

Readme syntax! — # content → it gets bold

2.) Status - displays the state of the code
git status

git status

File status lifecycle



3.) add → adds new or changed files in your working directory to the Git staging area.

git add <-file name->

git add. → means everything is staged

4.) commit - it is the record of change.

`git commit -m "some message"`

→ it is compulsory to write message.

5.) push - upload local repo content to remote repo.

`git push origin main`

* `mkdir name` (make directory) → command to create a folder

`cd name` → to go inside that folder

6.) init - used to create a new git repo (local)

`git init`

→ `git remote add origin <-link>`

→ `git remote -v` (to verify remote)

→ `git branch` (to check branch)

→ `git branch -M main` (to rename branch)

`git push origin main`

→ `git push -u origin main`

→ upstream

after that you can just write `git push`

→ if you have just modified the existing files then you can add & commit together

command: `git commit -am "message"`

* Workflow

GitHub

→ code changes

→ commit

local git

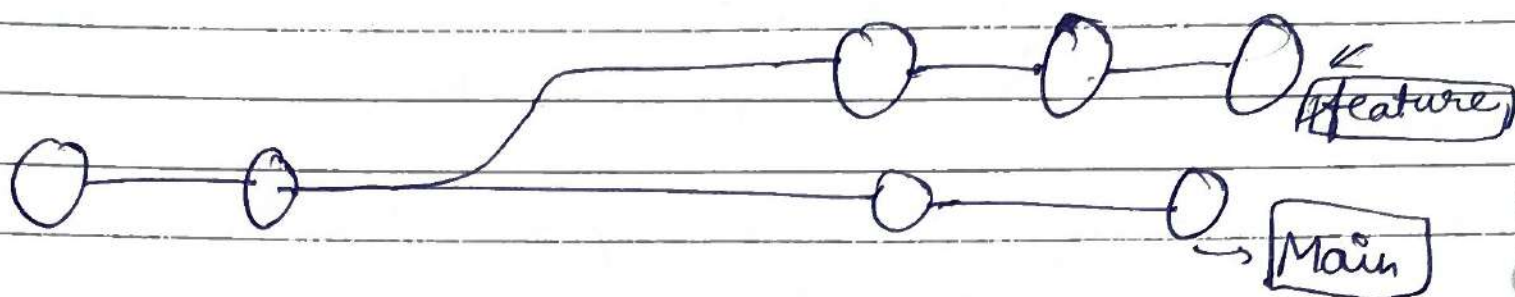
→ code changes

→ add (stage a change)

→ commit (commit changes)

→ push

★ Git Branches -



★ Branch Commands:-

- `git branch` (to check branch)
- `git branch -M (new name)` (to rename branch)
- `git checkout <branch name>` (to navigate)
- `git checkout -b <new branch name>` (to create new branch)
- `git branch -d <branch name>` (to delete branch)

★ To set upstream in new branch -

`git push --set-upstream origin feature`

★ Merging Code:-

- `git diff <branch name>` (to compare commits, branches, files & more)
- `git merge <branch name>` (to merge 2 branches)
or
PR (Pull request)

★ Pull Request:-

It lets you tell others about changes you have pushed to a branch in a repository on GitHub.

* `git pull origin main` → used to fetch & download content from a remote repo & immediately update the local repo to match that content.

* Merge Conflicts:-

An event that takes place when Git is unable to automatically resolve differences in code between two commits.

* fixing mistakes:-

1.) Staged changes -

`git reset <-file name->`

for all files last changes → `git reset`

2.) committed changes (for one commit)

`git reset HEAD~1`

→ it is a pointer which points towards last commit

* `git log` - into of all changes in reverse chronological order.

3.) committed changes (for many commits)

`git reset <commit hash->`

`git reset --hard <commit hash->`

* forking:-

A fork is a new repository that shares code & visibility settings with the original "upstream" repository.

* `git rm --cached *.class`

→ means remove from Git tracking only.

* `git pull origin main --allow-unrelated-histories`